Technical Specification Document: Focus Flow Productivity Suite

Project Name: Focus Flow

Version: 1.0 (MVP)

Objective: A unified dashboard combining Task Management (To-Do) and Time Optimization (Pomodoro) to help students manage their workloads and track their focus sessions.

---

1. Application Purpose & Scope

Focus Flow is designed to solve the common student problem of fragmented productivity. Instead of using separate apps for tasks and timers, Focus Flow integrates both. Users can create tasks and immediately initiate "Focus Sessions" linked to those specific tasks, allowing them to see exactly where their time is being spent.

---

2. Functional Modules

A. Task Management (The "To-Do" Engine)

- **Create:** Users can add tasks with a Title and a Priority level (Low, Medium, High).
- **Read:** A centralized list displays active tasks.
- **Update:** Users can toggle a task as "Complete" or "Active."
- **Delete:** Users can remove tasks from their list.

B. Time Optimization (The "Pomodoro" Engine)

- **Countdown Logic:** A standard 25-minute focus timer followed by a 5-minute break.
- **Controls:** Start, Pause, and Reset functionality.
- **Task Linking:** The timer must be linked to a selected task from the To-Do list to log data accurately.

C. Progress Tracking (The "Analytics" Engine)

- **Session History:** Logging completed focus sessions.
- **Summary:** Displaying total "Focus Time" achieved during the day.

---

3. UI/UX Design Requirements

- **Layout:** A single-page dashboard.
    - *Left Pane:* Task Management area.
    - *Right Pane:* Timer and Progress area.
- **Typography:** Global use of Plus Jakarta Sans.
- **Color Palette:** (Referencing the Branch Design System)
    - Background: bg-gradient-main (#030B22 → #02040A).
    - Primary Actions: bg-primary-blue (#1C4ED8).

    - Alerts/Timer Accents: text-primary-yellow (#FBBF24).
- **Deliverables:**
    - High-fidelity Figma Prototype.
    - Design System assets (Icons for Play/Pause, Delete, and Priority badges).
    - "Success State" animation for completed tasks.

---

4. Frontend Technical Requirements

- **State Management:** Maintain an array of Task objects.
- **Timer Logic:** Implement a robust JavaScript setInterval countdown that works accurately even if the user switches browser tabs.
- **Responsive Execution:** The dashboard must collapse into a stacked view (Timer on top, Tasks below) for mobile viewports.
- **Components to Build:**
    - TaskCard: Displays title, priority, and completion checkbox.
    - TimerDisplay: Large digital clock with a circular progress ring.
    - ControlGroup: Buttons for Start/Pause/Reset.

---

5. Backend Technical Requirements

- **Data Modeling (ERD):**
    - **Table: Tasks** (Fields: Id, Title, Priority, IsCompleted, CreatedAt).
    - **Table: FocusSessions** (Fields: Id, TaskId, StartTime, DurationSeconds, Status).
- **API Endpoints:**
    - GET /api/tasks: Retrieve all user tasks.
    - POST /api/tasks: Add a new task (Body: Title, Priority).
    - PATCH /api/tasks/{id}: Update task completion status.
    - DELETE /api/tasks/{id}: Remove a task.
    - POST /api/sessions: Log a completed focus session (Body: TaskId, Duration).
- **Documentation:** Provide a Swagger/OpenAPI definition or a clean README.md listing expected JSON request/response bodies for the Frontend team.

---

6. The "Hand-off" Workflow

1. **Design to Frontend:** UI/UX provides the Figma link and exported SVG icons.
2. **Backend to Frontend:** Backend provides the base URL and API documentation.
3. **Integration:** The Frontend team replaces local "Mock Data" with real fetch calls to the Backend endpoints.
4. **Version Control:** All code must be pushed to the designated GitHub repository using the branch's established **Code Conventions**.

---

7. Success Metrics

The application is considered "Production Ready" when:

1. A user can add a task.
2. A user can start a timer linked to that task.
3. Upon timer completion, the session is saved to the database.
4. The UI reflects the saved data upon page refresh.

Why Focus Flow? (Rationale for Selection)

The choice of a **To-Do List integrated with a Pomodoro Timer** as our central project was strategic. It provides the most effective "Learning Loop" for a 20-hour crash course (4 hours/day) for the following reasons:

- **Comprehensive Full-Stack Synergy:** Unlike simpler apps, Focus Flow requires all three sub-teams to succeed. The UI/UX team must solve the problem of visual clutter; the Frontend team must master real-time JavaScript state for the timer; and the Backend team must design a relational database to link time-logs to specific tasks.
- **The "CRUD" Industry Standard:** 90% of enterprise software is built on the **CRUD** (Create, Read, Update, Delete) pattern. By building the To-Do engine, students learn the fundamental architecture used by global tech companies.
- **Real-Time Logic Challenges:** Implementing a Pomodoro timer introduces the Frontend team to "Asynchronous Logic"—handling a countdown that continues accurately regardless of user interaction. This is a significant step up from static web pages.
- **Measurable Success for the Hackathon:** Focus Flow acts as a "mini-hackathon." It forces the teams to respect the **SRS** (Requirements) and the **Hand-off** timeline. Completing this project gives students the confidence and the "boilerplate" code needed to compete in the upcoming IEEE SVU SB Hackathon.
- **Utility & Relevance:** We are building a tool that we actually need. By creating a productivity app, we are solving a real problem for the students in the room, making the technical lessons feel immediate and purposeful.